



UNIVERSITÉ ABDELMALEK ESSAÂDI
ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES
AL-HOCEIMA

**Filière : Transformation Digitale et Intelligence
Artificielle 2**

Rapport TP 4

Applications web avancées avec Java et Spring

Gestion des Produits avec Architecture MVC, DAO et
Authentification

Réalisé par :

Moaad Daoudi

Encadré par :

Pr. Mohamed Cherradi

Année Universitaire 2025/2026

Table des matières

Introduction	3
1 Architecture Backend et Gestion de la Donnée	4
1.1 Modèles de Données (Models)	4
1.1.1 Classe <code>User.java</code>	4
1.1.2 Classe <code>Product.java</code>	4
1.2 La Couche d'Accès aux Données (DAO)	5
1.2.1 Interfaces DAO : <code>IProduct</code> et <code>IUser</code>	6
2 Logique Métier et Contrôleurs Web	8
2.1 Couche Métier (Services)	8
2.1.1 <code>ProductService</code> et <code>ProductServiceImpl</code>	8
2.2 Routage et C-R-U-D avec les Servlets	10
3 Interfaces Utilisateur (Vues) et Sécurité	14
3.1 Filtre et Sécurisation d'accès (<code>AuthFilter.java</code>)	14
3.2 Gestion des Vues	15
3.2.1 Espace Produit : <code>dashboard.jsp</code>	15
3.2.2 Authentification : <code>login.jsp</code> et <code>register.jsp</code>	16
3.2.3 Formulaire d'Ajout / Modification	17
Conclusion	19

Table des figures

1.1	Code source de l'entité <code>Product.java</code>	5
1.2	Méthodes d'accès base de données pour les Produits	7
2.1	Interface <code>ProductService.java</code> déclarant les opérations CRUD	9
2.2	Exposition du service produit avec <code>ProductServiceImpl.java</code>	10
2.3	Traitement d'ajout via la Servlet <code>AddProduitServlet</code>	11
2.4	Affichage et récupération via <code>ListProduitServlet</code>	12
2.5	Préparation de la modification via <code>EditProduitServlet</code> . . .	12
2.6	Mise à jour d'un produit existant via <code>UpdateProduitServlet</code>	13
2.7	Suppression d'un produit via la Servlet <code>DeleteProduitServlet</code>	13
3.1	Mécanisme d'interception et de protection implémenté par <code>AuthFilter</code>	15
3.2	Tableau de bord MVC1 affichant la liste des produits ainsi que la session de l'utilisateur	16
3.3	Interfaces d'authentification et de création de compte	17
3.4	Interface proposée pour l'ajout ou modification des produits .	18

Introduction

Contexte

Dans la continuité du TP précédent qui mettait en place l'authentification et la gestion des sessions HTTP, ce TP 4 vient enrichir notre application web basée sur Java Enterprise Edition (JEE) en introduisant un module complet de **Gestion des Produits**.

Objectifs du document

L'objectif de ce document est de présenter l'implémentation complète des fonctionnalités C-R-U-D (Création, Lecture, Mise à jour et Suppression) relatives aux Produits. Cette application web respecte scrupuleusement l'architecture MVC (Modèle-Vue-Contrôleur) et maintient une séparation des préoccupations forte via le pattern DAO pour l'accès aux données, tout en étant sécurisée grâce aux filtres d'authentification mis en place ultérieurement.

Chapitre 1

Architecture Backend et Gestion de la Donnée

1.1 Modèles de Données (Models)

En adéquation avec le principe de la programmation orientée objet, chaque entité stockable dans notre base est représentée au travers d'un Modèle (POJO).

1.1.1 Classe `User.java`

Issue de notre gestion d'authentification, la classe `User` modélise l'utilisateur dans l'application. Elle contient un `id`, `nom`, `email`, et `mot de passe`. Sa déclaration permet, via la base de données, la création de sessions HTTP une fois les correspondances email/mot de passe validées par les Servlets.

1.1.2 Classe `Product.java`

En parallèle de l'entité `User`, nous avons introduit la classe 'Product' représentant l'entité métier centrale de ce TP. Ce classe contient les attributs décrivant un produit : un identifiant unique (`id`), un `nom`, une `description` détaillée et un `prix`. C'est un Plain Old Java Object (POJO) standard, ce qui facilite son transit entre nos couches d'application.

A screenshot of a code editor with a dark background and light-colored text. The code is for a Java class named Product. It includes package declarations, private fields for id, nom, description, and prix, and public methods for getters and setters. The code is numbered from 1 to 54.

```
1 package ma.ensah.model;
2
3 public class Product {
4
5     private Long id;
6     private String nom;
7     private String description;
8     private double prix;
9
10    public Product() {
11        // TODO Auto-generated constructor stub
12    }
13
14    public Product(Long id, String nom, String description, double prix) {
15        this.id = id;
16        this.nom = nom;
17        this.description = description;
18        this.prix = prix;
19    }
20
21    public Long getId() {
22        return id;
23    }
24
25    public void setId(Long id) {
26        this.id = id;
27    }
28
29    public String getNom() {
30        return nom;
31    }
32
33    public void setNom(String nom) {
34        this.nom = nom;
35    }
36
37    public String getDescription() {
38        return description;
39    }
40
41    public void setDescription(String description) {
42        this.description = description;
43    }
44
45    public double getPrix() {
46        return prix;
47    }
48
49    public void setPrix(double prix) {
50        this.prix = prix;
51    }
52
53 }
54
```

FIGURE 1.1 – Code source de l'entité Product.java

1.2 La Couche d'Accès aux Données (DAO)

L'accès et la manipulation des produits et utilisateurs stockés en base de données sont assurés par les composants DAO respectifs : UserDaoImpl et ProductDaoImpl.

1.2.1 Interfaces DAO : IProduct et IUser

À l'instar de la gestion des utilisateurs, nous définissons un contrat via l'interface `IProduct.java` fournissant les méthodes nécessaires aux opérations CRUD :

- L'insertion d'un nouveau produit.
- La sélection d'un produit par ID et la récupération de l'ensemble des catalogues produit.
- La mise à jour de produits existants.
- La suppression par ID.

L'implémentation `JDBC ProductDaoImpl.java` prend en charge les `PreparedStatement` SQL (ex. `SELECT * FROM products`, `INSERT INTO...`) empêchant ainsi toute faille par Injection SQL tandis qu'il transforme chaque ligne de résultat (`ResultSet`) en de véritables objets Java manipulables par notre logique.

```
1 package ma.ensah.dao;
2
3 import java.util.ArrayList;
4 import java.util.List;
5 import java.util.Map;
6 import java.util.concurrent.ConcurrentHashMap;
7 import java.util.concurrent.atomic.AtomicLong;
8
9 import ma.ensah.model.Product;
10
11 public class ProductDaoImpl implements IProduct{
12
13     private static Map<Long, Product> productMap = new ConcurrentHashMap<>();
14     private static AtomicLong idCounter = new AtomicLong(1);
15
16     @Override
17     public void add(Product product) {
18         product.setId(idCounter.getAndIncrement());
19         productMap.put(product.getId(), product);
20     }
21
22     @Override
23     public void delete(Long id) {
24         productMap.remove(id);
25     }
26
27     @Override
28     public Product findById(Long id) {
29         return productMap.get(id);
30     }
31
32     @Override
33     public List<Product> findAll() {
34         return new ArrayList<>(productMap.values());
35     }
36
37     @Override
38     public void update(Product product) {
39         productMap.put(product.getId(), product);
40     }
41 }
42
43
```

FIGURE 1.2 – Méthodes d'accès base de données pour les Produits

Chapitre 2

Logique Métier et Contrôleurs Web

2.1 Couche Métier (Services)

2.1.1 ProductService et ProductServiceImpl

Ces classes agissent en tant que pont entre nos Servlets et la structure DAO. Leurs responsabilités couvrent l'ensemble du cycle de vie d'un produit (opérations CRUD) et l'application des règles de gestion :

- **L'ajout et la modification** : Validation des données du produit (nom, prix, et description) avant de finaliser l'insertion ou la mise à jour en base de données.
- **La lecture et recherche** : Récupération de l'ensemble du catalogue, ou d'un produit spécifique par son id (comme exploité par la barre de recherche du tableau de bord).
- **La suppression** : Transfert sécurisé de l'instruction de suppression vers la couche DAO.

Ce niveau d'abstraction assure un couplage faible, garantissant à l'application web (les Servlets) qu'elle ne communiquera jamais en direct avec les objets DAO.



```
1 package ma.ensah.service;
2
3 import java.util.List;
4
5 import ma.ensah.model.Product;
6
7 public interface ProductService {
8     void addProduct(Product product);
9     void deleteProduct(Long id);
10    Product getProduct(Long id);
11    List<Product> getAllProducts();
12    void updateProduct(Product product);
13 }
14
```

FIGURE 2.1 – Interface ProductService.java déclarant les opérations CRUD



```
1  package ma.ensah.service;
2
3  import ma.ensah.dao.IProduct;
4  import ma.ensah.dao.ProductDaoImpl;
5  import ma.ensah.model.Product;
6  import java.util.List;
7
8  public class ProductServiceImpl implements ProductService {
9
10     // We reference the Interface, not the concrete implementation
11     private IProduct productDao = new ProductDaoImpl();
12
13     @Override
14     public void addProduct(Product product) {
15         productDao.add(product);
16     }
17
18     @Override
19     public void deleteProduct(Long id) {
20         productDao.delete(id);
21     }
22
23     @Override
24     public Product getProduct(Long id) {
25         return productDao.findById(id);
26     }
27
28     @Override
29     public List<Product> getAllProducts() {
30         return productDao.findAll();
31     }
32
33     @Override
34     public void updateProduct(Product product) {
35         productDao.update(product);
36     }
37 }
```

FIGURE 2.2 – Exposition du service produit avec ProductServiceImpl.java

2.2 Routage et C-R-U-D avec les Servlets

La partie contrôleur est gérée par plusieurs Servlets ayant chacun une seule grande responsabilité pour satisfaire une composante du CRUD. Ainsi, le fichier `web.xml` est considérablement allégé des multiples actions ou le routage est géré purement par des annotations.

- `ListProduitServlet` : Affiche les informations récupérables en interrogeant le service et enchaîne le transfert vers la vue (JSP) adéquate

via `RequestDispatcher`.

- `AddProduitServlet` : Intercepte les formulaires d'ajout envoyés par requête `POST`, extrait et convertit les paramètres (Pensez de bien le paramètre du prix par `Double.parseDouble()`), puis les injectent dans un nouvel objet pour les enregistrer.
- `EditProduitServlet` / `UpdateProduitServlet` : Le premier réceptionne la demande d'édition et retourne un formulaire prérempli avec les données existantes, et le second traite les modifications postées de la couche vue.
- `DeleteProduitServlet` : Déclenchée lors d'une action "Supprimer" sur le tableau de bord, supprime la ressource via son identifiant `id`.



```
1 package ma.ensah.web;
2
3 import java.io.IOException;
4 import jakarta.servlet.ServletException;
5 import jakarta.servlet.annotation.WebServlet;
6 import jakarta.servlet.http.HttpServlet;
7 import jakarta.servlet.http.HttpServletRequest;
8 import jakarta.servlet.http.HttpServletResponse;
9 import ma.ensah.model.Product;
10 import ma.ensah.service.ProductService;
11
12 @WebServlet("/addProduit")
13 public class AddProduitServlet extends HttpServlet {
14     protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
15         ProductService service = (ProductService) getServletContext().getAttribute("productService");
16
17         String nom = request.getParameter("nom");
18         String desc = request.getParameter("description");
19         double prix = Double.parseDouble(request.getParameter("prix"));
20
21         service.addProduct(new Product(null, nom, desc, prix));
22         response.sendRedirect("listProduits");
23     }
24 }
```

FIGURE 2.3 – Traitement d'ajout via la Servlet `AddProduitServlet`

```
1 package ma.ensah.web;
2
3 import java.io.IOException;
4 import java.util.ArrayList;
5 import java.util.List;
6 import jakarta.servlet.ServletException;
7 import jakarta.servlet.annotation.WebServlet;
8 import jakarta.servlet.http.HttpServlet;
9 import jakarta.servlet.http.HttpServletRequest;
10 import jakarta.servlet.http.HttpServletResponse;
11 import ma.ensah.model.Product;
12 import ma.ensah.service.ProductService;
13
14 @WebServlet("/listProduits")
15 public class ListProduitServlet extends HttpServlet {
16
17     protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
18         ProductService service = (ProductService) getServletContext().getAttribute("productService");
19
20         String idParam = request.getParameter("idProduit");
21         List<Product> produits = new ArrayList<>();
22
23         if (idParam != null && !idParam.isEmpty()) {
24             try {
25                 Long id = Long.parseLong(idParam);
26                 Product p = service.getProduct(id);
27                 if (p != null) {
28                     produits.add(p); // On ajoute seulement le produit trouvé
29                 } else {
30                     request.setAttribute("error", "Produit introuvable avec l'ID : " + id);
31                     produits = service.getAllProducts(); // Optionnel : on affiche tout si non trouvé
32                 }
33             } catch (NumberFormatException e) {
34                 request.setAttribute("error", "Format d'ID invalide");
35                 produits = service.getAllProducts();
36             }
37         } else {
38             // Aucun paramètre : on affiche tout
39             produits = service.getAllProducts();
40         }
41
42         request.setAttribute("listeProduits", produits);
43         request.getRequestDispatcher("/dashboard.jsp").forward(request, response);
44     }
45 }
```

FIGURE 2.4 – Affichage et récupération via ListProduitServlet

```
1 package ma.ensah.web;
2
3 import java.io.IOException;
4 import jakarta.servlet.ServletException;
5 import jakarta.servlet.annotation.WebServlet;
6 import jakarta.servlet.http.HttpServlet;
7 import jakarta.servlet.http.HttpServletRequest;
8 import jakarta.servlet.http.HttpServletResponse;
9 import ma.ensah.service.ProductService;
10
11 @WebServlet("/editProduit")
12 public class EditProduitServlet extends HttpServlet {
13     protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
14         ProductService service = (ProductService) getServletContext().getAttribute("productService");
15         Long id = Long.parseLong(request.getParameter("id"));
16
17         request.setAttribute("productEdit", service.getProduct(id));
18         request.setAttribute("listeProduits", service.getAllProducts());
19         request.getRequestDispatcher("/dashboard.jsp").forward(request, response);
20     }
21 }
```

FIGURE 2.5 – Préparation de la modification via EditProduitServlet

```
1 package ma.ensah.web;
2
3 import java.io.IOException;
4 import jakarta.servlet.ServletException;
5 import jakarta.servlet.annotation.WebServlet;
6 import jakarta.servlet.http.HttpServlet;
7 import jakarta.servlet.http.HttpServletRequest;
8 import jakarta.servlet.http.HttpServletResponse;
9 import ma.ensah.model.Product;
10 import ma.ensah.service.ProductService;
11
12 @WebServlet("/updateProduit")
13 public class UpdateProduitServlet extends HttpServlet {
14     protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
15         ProductService service = (ProductService) getServletContext().getAttribute("productService");
16
17         Long id = Long.parseLong(request.getParameter("id"));
18         String nom = request.getParameter("nom");
19         String desc = request.getParameter("description");
20         double prix = Double.parseDouble(request.getParameter("prix"));
21
22         service.updateProduct(new Product(id, nom, desc, prix));
23         response.sendRedirect("listProduits");
24     }
25 }
```

FIGURE 2.6 – Mise à jour d'un produit existant via UpdateProduitServlet

```
1 package ma.ensah.web;
2
3 import java.io.IOException;
4 import jakarta.servlet.ServletException;
5 import jakarta.servlet.annotation.WebServlet;
6 import jakarta.servlet.http.HttpServlet;
7 import jakarta.servlet.http.HttpServletRequest;
8 import jakarta.servlet.http.HttpServletResponse;
9 import ma.ensah.service.ProductService;
10
11 @WebServlet("/deleteProduit")
12 public class DeleteProduitServlet extends HttpServlet {
13     protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
14         ProductService service = (ProductService) getServletContext().getAttribute("productService");
15         Long id = Long.parseLong(request.getParameter("id"));
16
17         service.deleteProduct(id);
18         response.sendRedirect("listProduits");
19     }
20 }
```

FIGURE 2.7 – Suppression d'un produit via la Servlet DeleteProduitServlet

Chapitre 3

Interfaces Utilisateur (Vues) et Sécurité

3.1 Filtre et Sécurisation d'accès (`AuthFilter.java`)

Avant de présenter les vues, il est impératif d'introduire le composant `AuthFilter`. Implémentant l'interface `Filter`, ce dernier se place comme un intercepteur de requêtes HTTP. Il vérifie l'existence d'une session valide avant d'autoriser l'accès aux pages JSP protégées ou aux contrôleurs sensibles (comme la gestion des produits). Si l'utilisateur n'est pas connecté, `AuthFilter` le redirige immédiatement vers la page d'authentification (`login.jsp`).



```
1 package ma.ensah.filters;
2
3 import jakarta.servlet.Filter;
4 import jakarta.servlet.FilterChain;
5 import jakarta.servlet.FilterConfig;
6 import jakarta.servlet.ServletException;
7 import jakarta.servlet.ServletRequest;
8 import jakarta.servlet.http.HttpServletRequest;
9 import jakarta.servlet.http.HttpServletResponse;
10 import jakarta.servlet.http.HttpSession;
11 import jakarta.servlet.ServletResponse;
12 import jakarta.servlet.annotation.WebFilter;
13
14 import java.io.IOException;
15
16 @WebFilter("/*")
17 public class AuthFilter implements Filter {
18
19     public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
20         throws IOException, ServletException {
21
22         HttpServletRequest req = (HttpServletRequest) request;
23         HttpServletResponse res = (HttpServletResponse) response;
24         String uri = req.getRequestURI();
25
26         boolean isIndex = uri.endsWith("/GestionDesProduits/") || uri.endsWith("index.jsp");
27         boolean isLoginPage = uri.endsWith("login.jsp");
28         boolean isRegisterPage = uri.endsWith("register.jsp");
29         boolean isLoginServlet = uri.endsWith("/login");
30         boolean isRegisterServlet = uri.endsWith("/register");
31
32         if (isLoginPage || isRegisterPage || isLoginServlet || isRegisterServlet || isIndex) {
33             chain.doFilter(request, response);
34             return;
35         }
36
37         HttpSession session = req.getSession(false);
38
39         if (session != null && session.getAttribute("user") != null) {
40             chain.doFilter(request, response);
41         } else {
42             res.sendRedirect("login.jsp");
43         }
44     }
45
46     public void init(FilterConfig fConfig) throws ServletException {}
47     public void destroy() {}
48 }
```

FIGURE 3.1 – Mécanisme d’interception et de protection implémenté par AuthFilter

3.2 Gestion des Vues

La composante visuelle qui touche au CRUD des produits n’est atteignable que par les utilisateurs reconnus au sein de la session courante.

3.2.1 Espace Produit : dashboard.jsp

Le composant `dashboard.jsp` agit comme vue principale pour la liste des produits. On y observe au sommet l’identité de l’utilisateur ("Welcome,

[Nom]") ainsi que son rôle (ex. "Role : USER"), accompagnés d'un bouton de "Logout" permettant de détruire la session. Au centre de l'interface, une barre de recherche permet de filtrer les résultats via un ID. Finalement, les données sont injectées dynamiquement dans un tableau HTML (ID, Nom, Description, Prix). L'utilisation des directives JSTL (JavaServer Pages Standard Tag Library) aide fortement à l'affichage d'une boucle des données.

Gestion des Produits (MVC1)

Welcome, daoudi moaad
Role: **ADMIN**
[Logout](#)

Ajouter un produit

Nom:

Description:

Prix:

Rechercher un produit

ID:

ID	Nom	Description	Prix	Actions
1	produit 1	produit1	11.0	Modifier Supprimer
2	produit 2	produit2	12.0	Modifier Supprimer
3	produit 3	produit3	13.0	Modifier Supprimer

FIGURE 3.2 – Tableau de bord MVC1 affichant la liste des produits ainsi que la session de l'utilisateur

3.2.2 Authentification : login.jsp et register.jsp

La sécurisation du tableau de bord ainsi que de toutes les servlets produit est rendue possible grâce au système d'authentification incluant `login.jsp` et `register.jsp`. Ces vues récupèrent les informations de l'utilisateur (nom, email, mot de passe) et les soumettent, à travers la méthode `POST`, aux servlets de la couche logique (`LoginServlet` et `RegisterServlet`). Une fois identifié, un objet session liera l'utilisateur au reste de son activité.

The figure displays two distinct user interface forms. The first form, titled 'Login to your app', features a title in bold black text, followed by two input fields labeled 'email' and 'mot de passe' (password). Below these fields is a 'login' button and a purple link that reads 'Register if you have an account'. The second form, titled 'Create an account', also has a bold black title and includes input fields for 'Nom' (Name), 'Prenom' (Surname), 'email', and 'mot de passe'. Additionally, it features a 'Role' dropdown menu currently set to 'Utilisateur' and a 'create account' button. A purple link labeled 'Login' is positioned at the bottom of this form.

Login to your app

email

mot de passe

[Register if you have an account](#)

Create an account

Nom

Prenom

Role

email

mot de passe

[Login](#)

FIGURE 3.3 – Interfaces d’authentification et de création de compte

3.2.3 Formulaire d’Ajout / Modification

Les actions de conception renvoient à des Vues secondaires qui proposent des champs simples et clairs. Chaque composant `<input>` assure, dans la vue, la récolte propre du Nom, de la Description et du Prix.

Ajouter un produit

Nom:

Description:

Prix:

FIGURE 3.4 – Interface proposée pour l'ajout ou modification des produits

Conclusion

La réalisation de ce TP 4 offre non seulement la concrétisation des concepts théoriques derrière le MVC, mais souligne également la modularité de l'application Java. L'intégration de la gestion métier d'entité "Produit", adossée au noyau d'authentification déjà réalisé, produit des pages hautement gérables et fonctionnelles. La gestion par DAO, l'emploi de modules de Service et des Servlets spécialisées permettent un projet extensible prêt pour la production.